

Phase 1 Technical Proposal & Statement of Work (SOW)

Executive Summary: Circle of Trust Trust & Policy Automation MVP

Document Date: October 8, 2025

Project Name: SG-X Guardian Client (On-Premise Core)

Phase Title: Phase 1: Circle of Trust & Policy Automation MVP (8 Weeks)

The objective of Phase 1 is to deliver a **Minimal Viable Product (MVP)** that establishes the foundational security architecture for the SG-X Guardian Client. This 8-week effort will build a cryptographically secure, self-hosted system where a cohort of devices can autonomously form a "**Circle of Trust**," synchronize a digitally signed security policy, and enforce it deterministically at the network layer. The architecture is explicitly **Self-Hosted First**, with clear provisions for future **Cloud-Optional** integration.

Value Delivered in Phase 1

Area	Value Proposition
Trust Fabric	Autonomous, Zero-Trust Fabric: Devices automatically discover peers, perform mutual software-based attestation , and establish secure mTLS channels.
Policy Control	Deterministic Policy Enforcement: A signed Universal Edge Processing (UEP) policy is atomically synchronized and enforced across the cohort using nftables for immediate L3/L4 security control.
Operational Readiness	Production-Readiness & Observability: System includes essential operational components: a tamper-evident audit log, Prometheus-style metrics, and a CLI status dashboard.
Future Path	Cloud-Optional Pathway: Implementation of a secure, outbound-only cloud uplink mock to validate the integration path for future cloud-based management.

2. Technical Architecture & Core Stack

The system comprises the core daemon, an administrative CLI tool, and internal microservices.

2.1. System Components

1. **sgx-guardian Daemon (Core Service):** Built in **Rust** and runs on every edge node. It contains the following logical services:
2. **p2p_discovery:** Uses **mDNS** for zero-configuration peer discovery.
3. **key_manager:** Manages the node's persistent **ECDSA P-256** identity key pair.
4. **attestation_service:** Facilitates mutual software-based attestation.
5. **policy_manager:** Verifies policy digital signatures and manages atomic installation/rollback.
6. **enforcement_engine:** Translates UEP policy rules into concrete **nftables** commands.
7. **telemetry_service:** Collects and exposes Prometheus-style system metrics.
8. **audit_logger:** Writes a structured, tamper-evident log of all critical security events.
9. **sgx-pa-cli Tool (Policy Authority CLI):** The on-premise command-line tool used by administrators for key pair generation, **policy signing**, and viewing local node status.

2.2. Core Technical Stack & Justification

Component	Technology Choice	Justification
Core Language	Rust	Memory safety, fearless concurrency, and high performance are non-negotiable for security.
P2P Communication	gRPC over mTLS	High-performance, strongly-typed RPC framework. mTLS ensures mutual authentication and encryption with forward secrecy.
Cryptography	ECDSA P-256	Standardized, efficient elliptic curve algorithm for FIPS-compliant device identity and policy signing.
Enforcement	nftables	Modern, kernel-level successor to iptables for deterministic, high-performance L3/L4 policy enforcement.

Component	Technology Choice	Justification
Packaging/Init	.deb/.rpm & systemd	Standard for enterprise Linux deployments, providing robust service lifecycle management.

2.3. Detailed P2P Trust Establishment Protocol

- 10. **Discovery:** Nodes use **mDNS** to broadcast a service type (`_sgx-guardian._tcp`) and a random `nonce`.
- 11. **Authentication:** Peers initiate a gRPC connection with a **mutual TLS (mTLS)** handshake, presenting a certificate signed by their persistent identity key.
- 12. **Attestation:** Nodes exchange an `AttestationEvidence` (a cryptographic signature over combined nonces and the active policy digest), proving **liveness** and binding identity to operational state.
- 13. **Virtual Identity:** The session's unique ID (`VirtualID`) is derived from a hash incorporating the device's public key, the policy digest, and the nonces, forcing re-attestation upon any state change.
- 14. **Policy Sync:** Policies are distributed via the secure gRPC channel. Receiving nodes **verify the Policy Authority's signature** before atomically installing the new policy and updating their `policy_digest`.

3. 8-Week Implementation Plan: Sprint Breakdown

The project is structured into four 2-week sprints, each with defined objectives and tangible, demonstrable deliverables.

Sprint	Objective	Key Tasks	Milestone Demo/Deliverable
Sprint 1 (Week 1-2) Foundation & Bootstrap	Establish project backbone and finalize core specifications.	Setup CI/CD pipeline and 3-node test LAN. Define all Protobuf API Schemas . Design the minimal UEP Policy Schema v1 (L3/L4 rules). Build <code>sgx-pa-cli</code> skeleton.	API Spec v1.0, UEP Schema v1.0, and configured dev environment.

Sprint	Objective	Key Tasks	Milestone Demo/Deliverable
Sprint 2 (Week 3-4) Identity & Discovery	Enable autonomous peer discovery and verifiable identity.	Implement mDNS p2p_discovery . Implement persistent key_manager (ECDSA P-256). Implement software attestation_service .	Milestone Demo 1: 3 nodes boot, discover each other, and successfully complete a mutual attestation handshake.
Sprint 3 (Week 5-6) Secure Channels & Policy Sync	Establish encrypted communication and secure policy distribution.	Implement mTLS over gRPC for secure channels. Complete sgx-pa-cli policy signing capability. Implement policy_manager for signature verification and atomic loading.	Milestone Demo 2: An admin signs a policy; it is securely distributed and verified by all peers in the cohort.
Sprint 4 (Week 7-8) Enforcement, Observability & Handoff	Activate enforcement, implement operational visibility, and finalize artifacts.	Implement enforcement_engine (UEP v1 to nftables translator). Implement telemetry_service (metrics) and audit_logger (tamper-evident log). Implement Cloud Uplink Mock (secure outbound-only channel). Create .deb/.rpm packages and final documentation.	Final Demo: Full E2E script proving trust formation, policy sync, and active L3/L4 enforcement, with status visibility via the CLI.

4. Team Structure & Responsibilities (3 FTE)

Role	Key Responsibilities
Rust Backend Engineer	P2P protocol state machine, gRPC/mTLS, cryptographic routines, core daemon architecture, and identity management.
Linux Systems Engineer	nftables integration, policy engine logic, OS packaging (.deb/.rpm), systemd hardening, and telemetry/audit pipeline.
Project Lead / DevOps Engineer	Project management, sprint coordination, CI/CD, comprehensive integration testing, documentation synthesis, and client demonstration.

5. Phase 1 Deliverables (Tangible Outcomes)

5. Source Code: A complete, modular, and documented Rust codebase for `sgx-guardian` and `sgx-pa-cli`.

6. Design Artifacts:

7. `API_Specification_v1.0.pdf` (Protobuf schemas & sequence diagrams).

8. `UEP_Policy_Schema_v1.0.pdf` (YAML/JSON structure).

9. Working Software & Artifacts:

10. `sgx-guardian` daemon binary.

11. `sgx-pa-cli` administration tool.

12. `.deb` and `.rpm` installation packages with `systemd` configuration.

13. Documentation:

14. `Administrator_Guide_v0.5.pdf` (Installation, configuration, key management, and operational procedures).

15. `Test_Report.pdf` (Summary of all unit and integration test results).

16. **Demonstration:** A repeatable, scripted demo proving the core value proposition on a 3-node cohort.

6. Scope Exclusions: Deferral for Phase 2+

To ensure timely delivery of a robust foundation, the following complex features from the original SOW are explicitly deferred to subsequent phases.

Feature	Status in Phase 1	Justification for Deferral
Virtual Shift (AI/ML)	✗ Deferred	Requires significant R&D; the Circle of Trust plane must be stable first.
Protocol Connectors	✗ Deferred	L7 protocol parsing (Modbus, OPC-UA) is a substantial, separate engineering effort.
Hardware TPM Attestation	✗ Deferred	Software attestation validates the protocol flow. TPM integration adds hardware-specific complexity.
Advanced Enforcement	✗ Deferred	nftables is sufficient for the MVP. eBPF/DPDK is a performance optimization for later.
CRL Gossip & Advanced Audit	✗ Deferred	A full Certificate Revocation List (CRL) gossip protocol is a complex distributed systems problem.
Bi-directional Cloud Control	✗ Deferred	Phase 1 is Self-Hosted First . Only a secure, outbound-only channel is implemented for future readiness.